



Photo by jggrz on pixabay

Lecture 9 – Boosting

Taubenschwänzchen (*Macroglossum stellatarum*)

- ♦ Ein sehr außergewöhnlicher Schmetterling ist das Taubenschwänzchen, ein (tagaktiver) Nachtfalter aus der Familie der Schwärmer.
- ♦ Sein deutscher Name leitet sich vom zweigeteilten Haarbüschel am Hinterleibsende ab, das eine gewisse Ähnlichkeit mit den Schwanzfedern von Tauben aufweist.
- ♦ Die Falter erreichen eine Flügelspannweite von 36 bis 76 Millimetern bei einer Masse von etwa 0,3 Gramm. Taubenschwänzchen haben einen etwa drei Zentimeter langen Rüssel, der es ihnen ermöglicht, im Flug zu trinken. Dabei wählen sie vor allem Blüten mit langem Kelch.
- ♦ Wegen seines auffälligen Flugverhaltens, das als Schwirrflug bezeichnet wird und das dem eines Kolibris ähnelt, wird es auch Kolibrischwärmer genannt. Zahlreiche vermeintliche Kolibrisichtungen in Europa gehen auf Beobachtungen an dieser Schmetterlingsart zurück.

Quellen:

* <https://de.wikipedia.org/wiki/Taubenschwänzchen>

* <https://www.plantura.garden/gruenes-leben/schmetterlinge-die-10-beliebtesten-heimischen-schmetterlingsarten>



Boosting

1. Boosting
2. AdaBoost
3. Gradient Boosting for Regression
4. Gradient Boosting for Classification



Boosting – Main Idea

♦ Boosting

- “*why train only on decision tree, when you can have many*”
- Train a sequence of models
- Focus later models on the observations, that earlier models get wrong („the hard ones“)
- Use relatively „simple“ models (called weak learners) = high-bias/low variance models
- Combine multiple high-bias/low-variance Decision Trees
- Finally: Combine models

➔ Convert many simple (high-bias) models into one complex classifier

➔ Avoid underfitting by reducing bias

➔ Avoid overfitting by controlling variance

➔ Ensemble Learning Technique



Boosting

1. Boosting
2. AdaBoost
3. Gradient Boosting for Regression
4. Gradient Boosting for Classification



AdaBoost

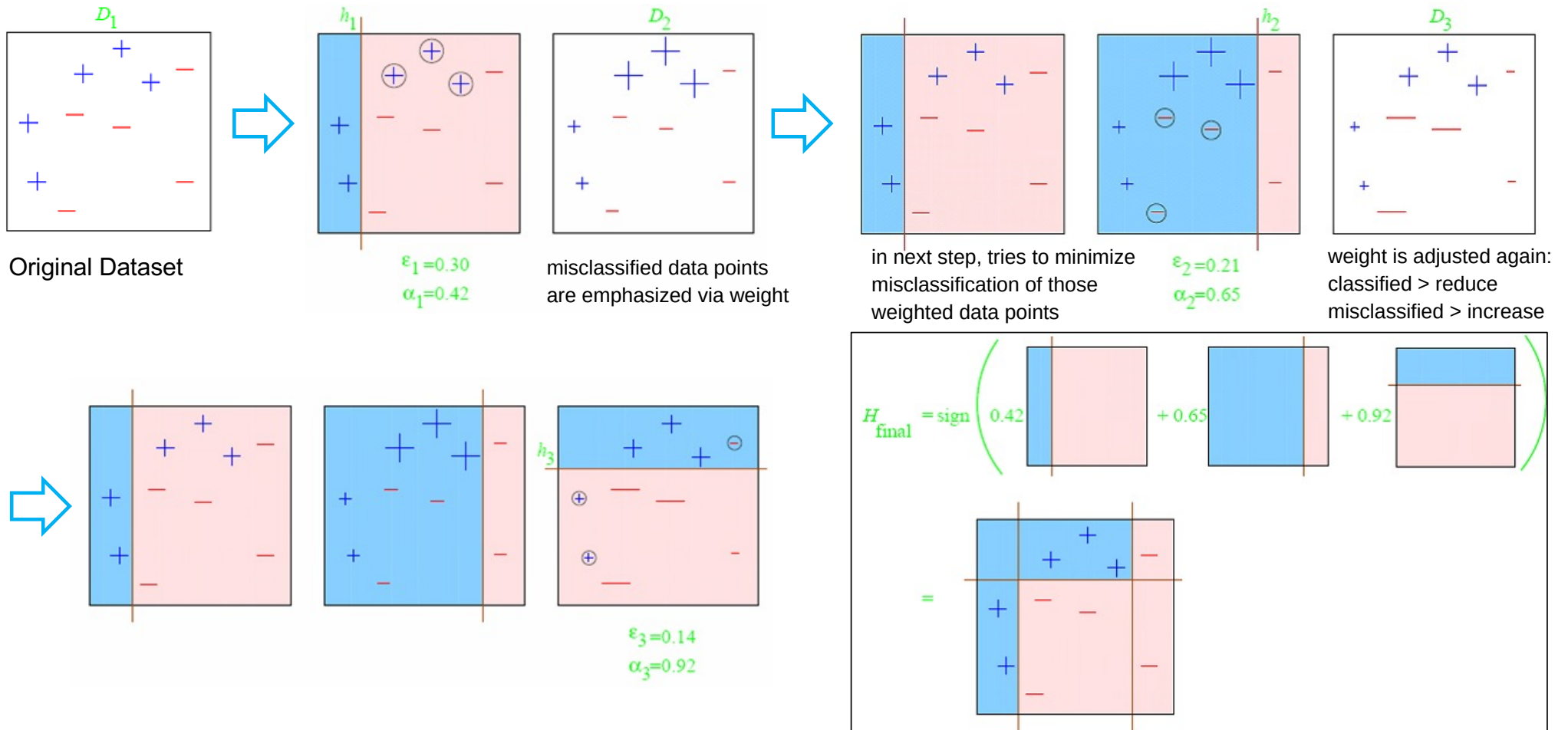
- ◆ “When nothing works, Boosting does”.
 - Many people use advanced boosting-based decision trees (like XGBoost or LightGBM or CatBoost) to win competitions at Kaggle or Hackathons.
 - AdaBoost was/is the first steppingstone into the world of Boosting.
 - Gradient Boosting is a generalization of AdaBoost.
 - “Extreme” gradient boosting algorithms are state-of-the-art → later lecture

- ◆ AdaBoost (Adaptive Boosting) [Freund, Schapire, 1997]
 - Iteratively add classifiers, each time reweighting the dataset to focus the next classifier on where the current one makes errors.
 - Weak learners: decision stumps, i.e., decision trees with a single split
 - Put more weight on difficult to classify instances and less on those already handled well.
 - Can be used for both classification and regression problem.



AdaBoost – Example (from Schapire's tutorial) – using Decision Stumps

Source/based on: <https://alliance.seas.upenn.edu/~cis520/wiki/index.php?n=Lectures.Boosting>



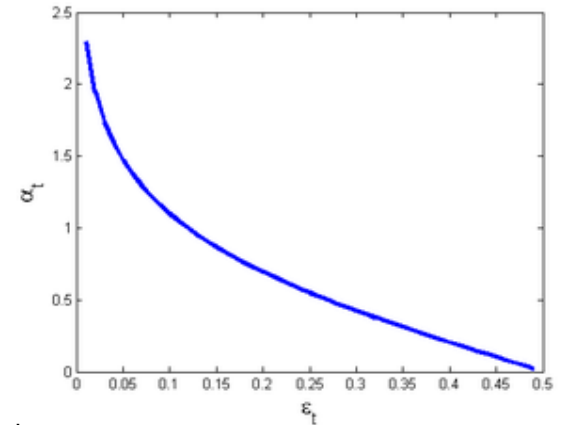
number of iterations is hyperparam, either max num iterations or epsilon difference becomes negligent
can only split parallel to axis



AdaBoost - Algorithm

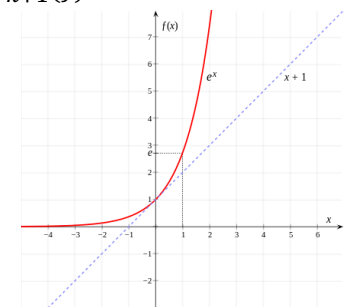
◆ AdaBoost Algorithm

- Given: n examples $X = \{\vec{x}_i\}$, $Y = \{y_i\}$, where $y_i \in \pm 1$, $i = 1 \dots n$
- Initialize: weights $w_1(i) = \frac{1}{n}$
- For $k = 1 \dots K$
 - Train simple classifier: $C_k = \text{train}(X, Y, w_k)$
 - Compute predictions: $\hat{Y} = C_k.\text{predict}(X)$
 - Compute weighted error: $\varepsilon = \sum_{i=1}^n w_k(i) \cdot (y_i \neq \hat{y}_i)$ error is sum of weights of misclassified data points
 - Compute new coefficients: $\alpha_k = \frac{1}{2} \cdot \log\left(\frac{1-\varepsilon}{\varepsilon}\right)$ i.e. effectiveness of classifier, the higher the better
 - Compute new weights: $w_{k+1}(i) = w_k(i) \cdot e^{-\alpha_k y_i \hat{y}_i}$ and normalize them $w_{k+1}(i) = \frac{w_{k+1}(i)}{\sum_{j=1}^n w_{k+1}(j)}$
- Combined classifier: $C = \text{sign}(\sum_{k=1}^K \alpha_k C_k)$



◆ Why exactly these choices for α_k and $w_{k+1}(i)$?

- Nice theoretical background – minimizes exponential loss function
- Details see e.g. <https://en.wikipedia.org/wiki/AdaBoost>

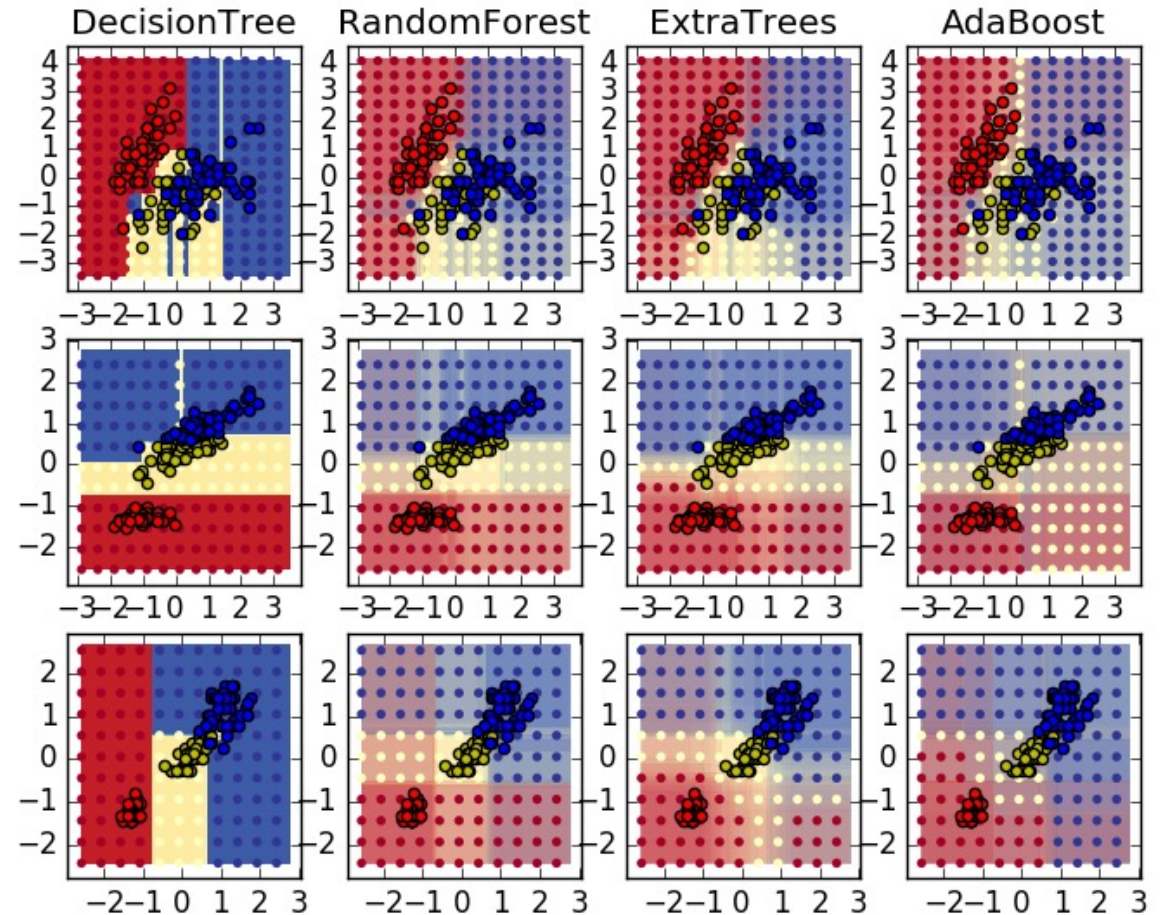


◆ Weighted Decision Tree: use weights instead of probabilities for split criterion (e.g. IG)



Random Forests, ExtraTrees and AdaBoost Example

- ♦ Example: Combinations of predictors for the Iris dataset:
 - Sepal Length vs. Sepal Width
 - Sepal Length vs. Petal Length
 - Petal Length vs. Petal Width.



source: http://scikit-learn.org/stable/auto_examples/ensemble/plot_forest_iris.html



Exercises

Exercise 1

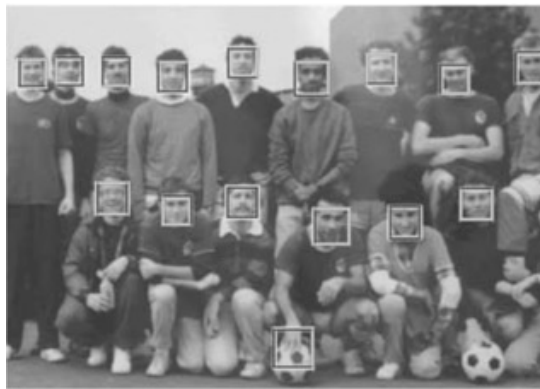
Weighted Decision Trees





AdaBoost – Practical Example

- ◆ Face Detection using the **Viola-Jones Algorithm** [Viola, Jones, 2001]
 - Detects faces in images in real-time
 - Slides a Haar-Wavelet of the image for feature extraction (>100k features)
 - AdaBoost for classification of windows that contain faces
 - Lots of other tricks and optimizations → See „Bildverarbeitung“ for all the details!



Source/based on: <https://alliance.seas.upenn.edu/~cis520/wiki/index.php?n=Lectures.Boosting>

- Fast enough for videos: <https://www.youtube.com/watch?v=22m80cXDok0>



Boosting

1. Boosting
2. AdaBoost
3. Gradient Boosting for Regression
4. Gradient Boosting for Classification



Gradient Boosting

◆ Gradient Boosting

- Train a sequence of models
- Focus later models on “shortcomings” of the earlier models
 - Gradient Boosting: Shortcomings identified by the gradient
 - AdaBoost: shortcomings identified by high-weight training examples
- Use relatively „simple“ models
- Finally: Combine models

not as fast as random forest since the training is done in sequence

◆ Basic Approach

- Start with a constant model (“one node tree”)
- Iterate
 - Fit a (relatively simple) model to the negative gradient of the loss function of the previous
 - Take a step into this direction by combining the previous model with the new model



Example and Initialization

- ♦ Initialization with a “constant” learner using the average target

71.2

Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57



Iteration step 1: compute pseudo residuals

71.2

◆ Compute Pseudo Residuals

- difference between the observed value and the predicted value based on the previous iteration (in the first iteration, this is always 71.2)

Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	16.8
1.6	Green	Female	76	4.8
1.5	Blue	Female	56	-15.2
1.8	Red	Male	73	1.8
1.5	Green	Male	77	5.8
1.4	Blue	Female	57	-14.2

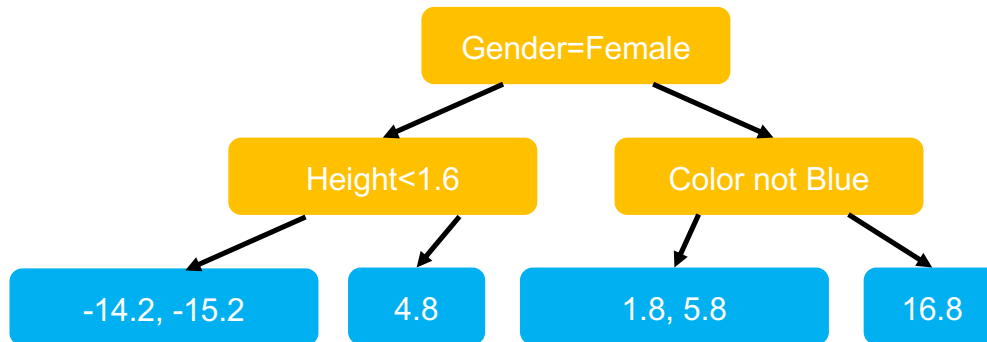


Iteration step 1: build new model (tree)

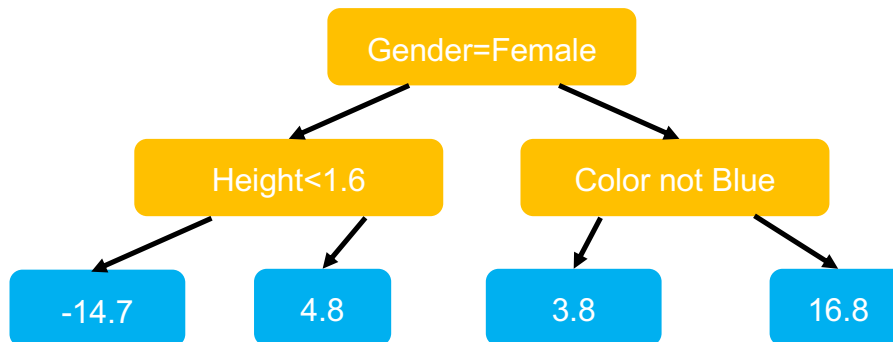
these trees are regression trees and are built as follows:
for each feature do
 for each threshold
 calc sum of squared residuals
 choose threshold with lowest SSR compared to other thresholds
choose feature with lowest SSR in comparison to other features

SSR is calculated for prediction target (here: weight)

- Build a tree based on the errors of the previous step (parameter ***max_depth***=2 here, default 3)



- Replace multiple entries in the leaves with mean



Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	16.8
1.6	Green	Female	76	4.8
1.5	Blue	Female	56	-15.2
1.8	Red	Male	73	1.8
1.5	Green	Male	77	5.8
1.4	Blue	Female	57	-14.2

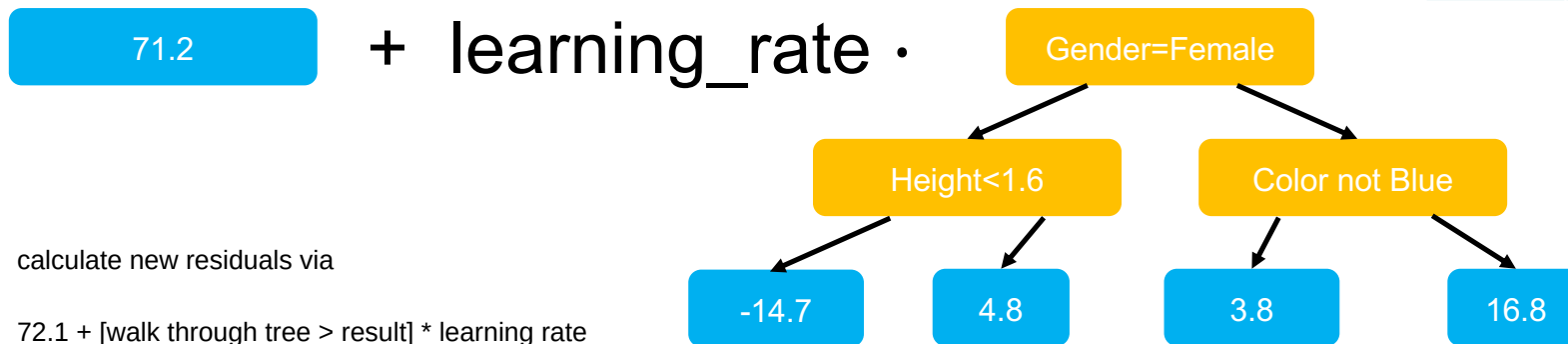
for gradient boosting, instead of using weight, SSR is calculated for the pseudo residuals so that it improves on the errors of the previous model



Iteration step 1: combine the models

Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

- Combine the previous model with the new model using a **learning_rate** (parameter, between 0 and 1, default 0.1) to control overfitting



calculate new residuals via

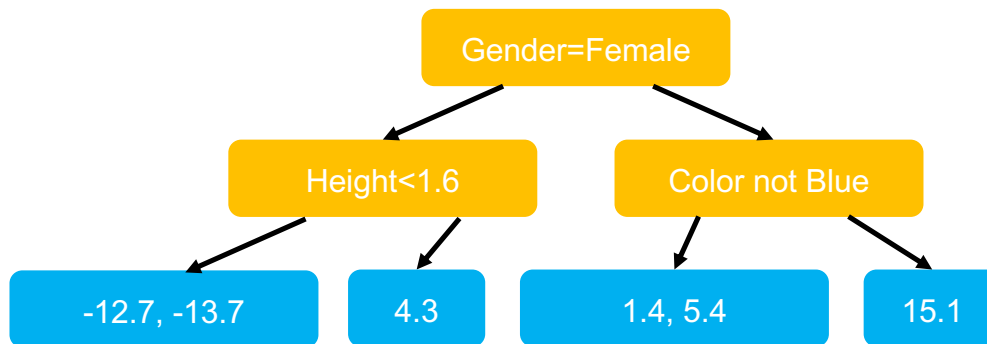
$72.1 + [\text{walk through tree} > \text{result}] * \text{learning rate}$

- E.g. the first data points: observed weight = 88
 - Predicted for learning_rate = 1.0 (no scaling): $71.2 + 16.8 = 88 \rightarrow$ too good, overfitting
 - Predicted for learning_rate = 0.1: $71.2 + 0.1 * 16.8 = 72.9 \rightarrow$ not as good, but better than before
 - Small step in the right direction
 - Empirical results: many such small step lead to lower variance (i.e., better test accuracy)

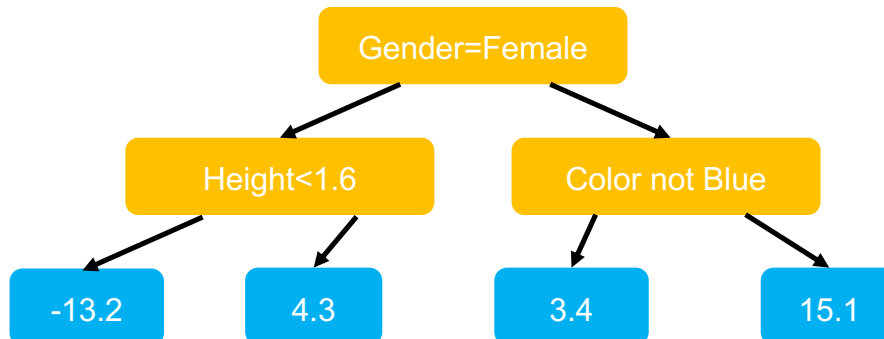


Iteration step 2: compute pseudo residuals and build new model (tree)

- ◆ Compute new Pseudo Residuals and build next tree



- ◆ Replace multiple entries with their mean



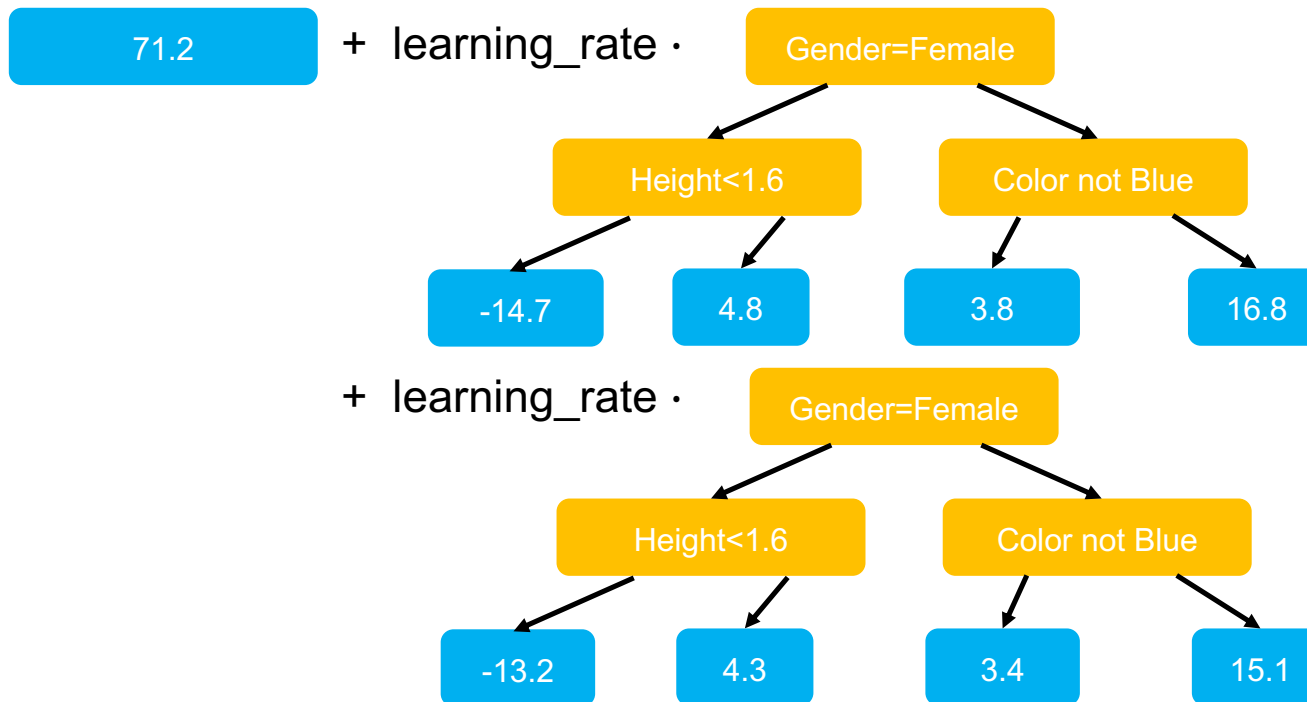
Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	15.1
1.6	Green	Female	76	4.3
1.5	Blue	Female	56	-13.7
1.8	Red	Male	73	1.4
1.5	Green	Male	77	5.4
1.4	Blue	Female	57	-12.7

usually does not result in the same tree
sum of pseudo residuals becomes smaller
(or else it would not terminate)



Iteration step 2: combine the models

Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57



- ♦ E.g. the first data points: observed weight = 88
 - Predicted for learning_rate = 0.1: $71.2 + 0.1 \cdot 16.8 + 0.1 \cdot 15.1 = 74.4$
 - Another small step in the right direction (from 72.9 to 74.4)



Stopping Criterion

- ◆ Keep adding new trees until
 - maximum number of trees reached (parameter ***n_estimators***, default 100)
 - additional trees does not significantly reduce the size of the residuals (parameter ***tol***, default *1e-4*)
- ◆ Use the final model to predict target for new data points
- ◆ The given example uses “squared error” as loss function, other loss functions can be used
 - Squared error is the most frequently used one for regression



Gradient Boosting – Full Algorithm

- ◆ Input: training data $\mathbf{x}_i, y_i$ ($i = 1, \dots, n$) and a differentiable Loss Function $L(y_i, F(\mathbf{x}))$
- ◆ Step 1: Initialize with a constant value: $F_0(\mathbf{x}) = \underset{\gamma}{\operatorname{argmin}} \sum_{i=1}^n L(y_i, \gamma)$
- ◆ Step 2: for $m = 1, \dots, n_estimators$
 - (a) Compute $r_{im} = - \left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F(\mathbf{x})=F_{m-1}(\mathbf{x})}$ for $i = 1, \dots, n$
 - (b) Fit regression tree to the r_{im} and create “terminal regions” R_{jm} for $j = 1, \dots, J_m$
 - (c) For $j = 1, \dots, J_m$ compute $\gamma_{jm} = \underset{\gamma}{\operatorname{argmin}} \sum_{\mathbf{x}_i \in R_{ij}} L(y_i, F_{m-1}(\mathbf{x}_i) + \gamma)$
 - (d) Update $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu \sum_{j=1}^{J_m} \gamma_{jm} I(\mathbf{x} \in R_{jm})$
- ◆ Step 3: output $F_M(\mathbf{x})$



Gradient Boosting – Full Algorithm in Detail (1)

- ◆ **Input: training data x_i, y_i ($i = 1, \dots, n$) and a differentiable Loss Function $L(y_i, F(x))$**

- Most frequently used loss function for regression:

squared error: $L(y_i, F(x)) = \frac{1}{2} (y_i - F(x))^2$

with the derivate
of the loss function $\frac{\partial L(y_i, F(x))}{\partial F(x)} = \frac{\partial}{\partial F(x)} \frac{1}{2} (y_i - F(x))^2 = -(y_i - F(x))$

- ◆ **Step 1: Initialize with a constant value: $F_0(x) = \underset{\gamma}{\operatorname{argmin}} \sum_{i=1}^n L(y_i, \gamma)$**

- For the squared error loss function:

Compute the derivate, set to 0 and solve:

$$\frac{\partial}{\partial \gamma} \sum_{i=1}^n L(y_i, \gamma) = \sum_{i=1}^n -(y_i - \gamma) = -(\sum_{i=1}^n y_i) + n\gamma$$

set to 0: $-(\sum_{i=1}^n y_i) + n\gamma = 0$

and solve: $\gamma = \frac{1}{n} \sum_{i=1}^n y_i$

- Thus $F_0(x) = \frac{1}{n} \sum_{i=1}^n y_i$ i.e. first step: average of prediction target



Gradient Boosting – Full Algorithm in Detail (2)

♦ Step 2: for $m = 1, \dots, n_estimators$

(a) Compute $r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)}$ for $i = 1, \dots, n$

- For the squared error loss function, these are the residuals
i.e. leaves in tree

(b) Fit regression tree to the r_{im} and create “terminal regions” R_{jm} for $j = 1, \dots, J_m$

- The leaves of the tree are the terminal regions, J_m is the number of leaves of the m th tree

(c) For $j = 1, \dots, J_m$ compute $\gamma_{jm} = \underset{\gamma}{\operatorname{argmin}} \sum_{x_i \in R_{ij}} L(y_i, F_{m-1}(x_i) + \gamma)$

- Compute an output value for each leaf j in the new tree. This is similar to step 1, but we take the previous prediction into account, include only the training examples in this leaf. For the squared error loss function, this becomes the average of the residuals that end up in this leaf.

(d) Update $F_m(x) = F_{m-1}(x) + \nu \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$

- Compute the new output value of the model, by taking the previous output value and adding in the prediction of the new model weighted by the learning rate ν . The sum is needed if a training examples ends up in multiple leaves: we add up the output values γ_{jm} for all the leaves R_{jm} that x ended up in.



Boosting

1. Boosting
2. AdaBoost
3. Gradient Boosting for Regression
4. Gradient Boosting for Classification

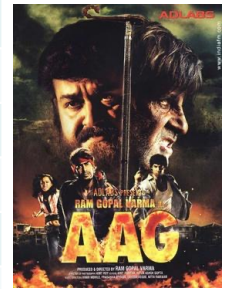


Example and Initialization

- ♦ Binary Classification
- ♦ Idea: use Gradient Boosting on the Regression by using the probability of the positive class as target
- ♦ Initialization with a “constant” learner using the $\log(\text{odds})$

$$\ln(4/2) \approx 0.6931$$

Likes Popcorn	Age	Favorite Color	Loves Aag
Yes	12	Blue	Yes
Yes	87	Green	Yes
No	44	Blue	No
Yes	19	Red	No
No	32	Green	Yes
No	14	Blue	Yes



Source: example data adapted from StatQuest/Josh Stamer
See also: [https://en.wikipedia.org/wiki/Aag_\(2007_film\)](https://en.wikipedia.org/wiki/Aag_(2007_film))



Odds vs. Probabilities



- ♦ **Odds (de: Chance / Quote):** Ratio of something happening to something not happening

$$\text{odds} = \frac{\text{something happening}}{\text{something not happening}}$$

- ♦ **Probability:** Ratio of something happening to everything that could happen

$$\text{probability} = \frac{\text{something happening}}{\text{everything that could happen}}$$

- ♦ Example: The Starbulls won 5 games and lost 3 games in the last month: ●●●●●●●●

- **Odds:** 5 to 3:
$$\text{odds} = \frac{\text{Starbulls winning}}{\text{Starbulls losing}} = \frac{\text{●●●●●}}{\text{●●●}} = \frac{5}{3} \approx 1.7$$

- **Probability:**
$$p = \frac{\text{Starbulls winning}}{\text{number of games}} = \frac{\text{Starbulls winning}}{\text{Starbulls winning or losing}} = \frac{\text{●●●●●}}{\text{●●●●●●●●}} = \frac{5}{5+3} = 0.625$$



Conversions of Odds and Probabilities

- ◆ Number of trials: n ; number of wins W and of losses of L (so $n = W + L$)

- Odds in favour and against: $o_f = \frac{W}{L}$ and $o_a = \frac{L}{W}$ (therefore, $o_f \cdot o_a = 1$)
- probability of winning: $p = \frac{W}{n} = \frac{W}{W+L}$
- probability of losing: $q = \frac{L}{n} = (1 - p)$

- ◆ Probabilities to Odds

- Odds in favour are equal to the Probability of winning divided by the probability of losing:

$$o_f = \frac{p}{1-p}$$

$$\text{Proof: } \frac{p}{(1-p)} = \frac{W/n}{L/n} = \frac{W}{L}$$

➔ Odds can be calculated from counts or from probabilities (both used in practise)

- ◆ Odds to Probabilities

solve above equation for p to get this formula

$$p = \frac{o_f}{o_f + 1}$$

and

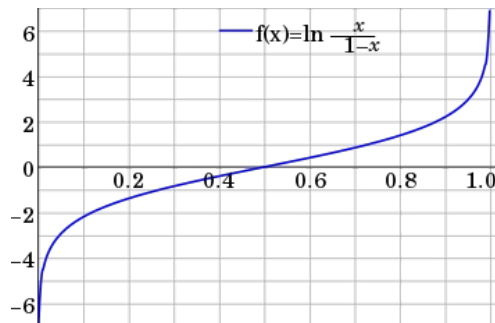
$$q = \frac{1}{o_f + 1}$$



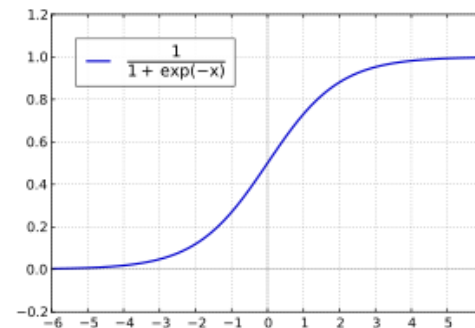
log(odds) – logit and sigmoid Functions

- ♦ Often, an unbounded range of $[-\infty, \infty]$ is easiest to handle $\rightarrow \log(odds)$
 - Range of Probabilities: $[0, 1]$
 - Range of Odds: $[0, \infty]$
 - Range of odds that are in favor ($p > 0.5$) is $(1, \infty]$ and against ($p < 0.5$) is $[0, 1)$ $\rightarrow$ highly unbalanced
 - Range of $\log(odds)$: $[-\infty, \infty]$
 - Range of $\log(odds)$ that are in favor ($p > 0.5$) is $(0, \infty]$ and against ($p < 0.5$) is $[-\infty, 0)$ $\rightarrow$ nicely balanced

- ♦ Logit function: $\text{logit}(p) = \ln\left(\frac{p}{1-p}\right)$ with it's inverse: logistic function/sigmoid $\text{sig}(t) = \frac{1}{1+e^{-t}} = \frac{e^t}{1+e^t}$



Logit-Function



Sigmoid-Function

Side Note:
Logistic Regression =
use linear Regression on
the logit in order to do
Classification



Iteration step

♦ Compute Pseudo Residuals

- Difference between the observed value and the predicted value based on the previous iteration
- Based on the probability
- Convert log(odds) into probabilities with the sigmoid function:

$$\text{sig}\left(\ln\frac{4}{2}\right) = \frac{e^{\ln(4/2)}}{1+e^{\ln(4/2)}} \approx 0.6667 \approx 0.7$$

- Note: if we classify as "yes" for > 0.5 , we will classify everyone as loving Aag

Likes Popcorn	Age	Favorite Color	Loves Aag	Residual
Yes	12	Blue	Yes	0.3
Yes	87	Green	Yes	0.3
No	44	Blue	No	-0.7
Yes	19	Red	No	-0.7
No	32	Green	Yes	0.3
No	14	Blue	Yes	0.3

Loves Aag values are converted to numeric via yes = 1 and no = 0

residuals are computed by subtracting from those values

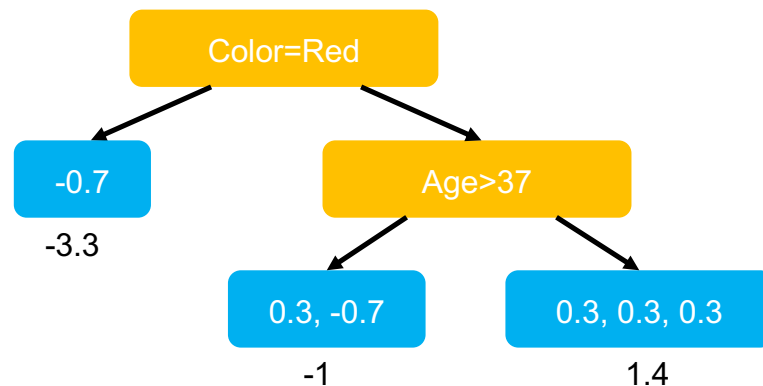


Iteration step 1: build new model (tree)

- Build a tree based on the errors of the previous step (here with parameter ***max_leaf_nodes***=3)

Reminder: Previous "Tree"

$$\ln(4/2) \approx 0.6931$$



Likes Popcorn	Age	Favorite Color	Loves Aag	Residual
Yes	12	Blue	Yes	0.3
Yes	87	Green	Yes	0.3
No	44	Blue	No	-0.7
Yes	19	Red	No	-0.7
No	32	Green	Yes	0.3
No	14	Blue	Yes	0.3

- Calculate output values for the new tree
 - Note: values in the tree are „probability residuals“, but we need „log(odds)“ output values

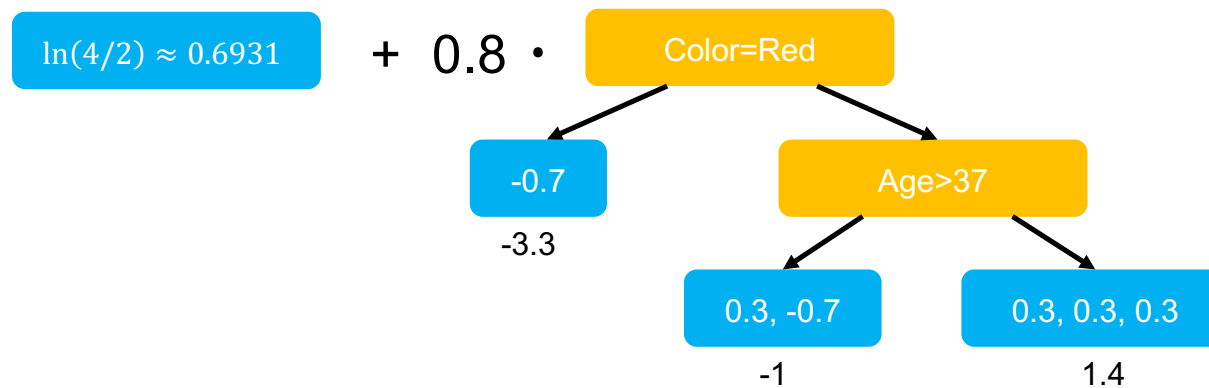
→ Transform using
$$\frac{\sum_i residual_i}{\sum_i [predictedProbability_i \cdot (1 - predictedProbability_i)]}$$

- E.g., for the middle leaf:
$$\frac{0.3 + -0.7}{0.7 \cdot (1 - 0.7) + 0.7 \cdot (1 - 0.7)} \approx -1$$



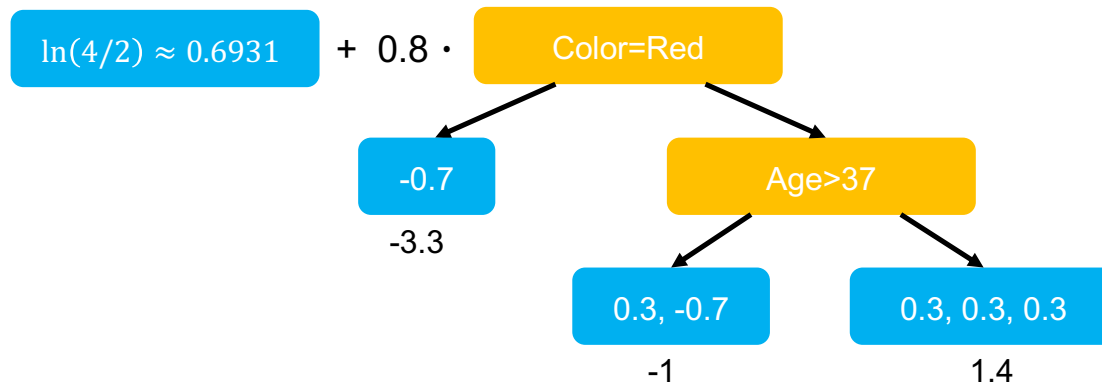
Iteration step 1: combine the models

- Combine the previous model with the new model using the **learning_rate** (default 0.1, here: 0.8) to control overfitting





Iteration step 2: compute new predicted probabilities and pseudo residuals



◆ Compute

- the new predicted log(odds) from the combined tree,
- convert to probabilities and compute pseudo residuals
- Example for the first training data point

- Log(odds) prediction $\approx 0.6931 + 0.8 \cdot 1.4 \approx 1.8131$
- Predicted probability $\approx \text{sig}(1.8) = \frac{e^{1.8}}{1+e^{1.8}} \approx 0.8597$

- Example for the second training data point

- Log(odds) prediction $\approx 0.6931 + 0.8 \cdot -1 = -0.1069$
- Predicted probability $\approx \text{sig}(-0.1069) = \frac{e^{-0.1069}}{1+e^{-0.1069}} \approx 0.4733$

Likes Popcorn	Age	Favorite Color	Loves Aag	Pred. Prob.	Residual
Yes	12	Blue	Yes	0.8597	0.1403
Yes	87	Green	Yes	0.4733	0.5267
No	44	Blue	No	0.4733	-0.5267
Yes	19	Red	No	0.1249	-0.1249
No	32	Green	Yes	0.8597	0.1403
No	14	Blue	Yes	0.8597	0.1403

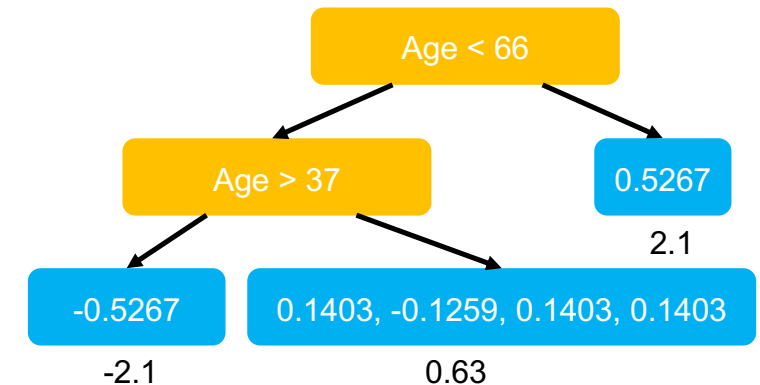


Iteration step 2: build new model (tree) and compute output values

- Using the new residuals, build a new tree
- Calculate the output values using the transformation

$$\frac{\sum_i residual_i}{\sum_i [predictedProbability_i \cdot (1 - predictedProbability_i)]}$$

- e.g., for the left leaf (2nd data point) : $\frac{-0.5267}{0.4733 \cdot (1 - 0.4733)} \approx -2.1$
- e.g., for the right leaf (3rd data point): $\frac{0.5267}{0.4733 \cdot (1 - 0.4733)} \approx 2.1$
- e.g., for the middle leaf (all other data points):
 $\frac{0.1403 + (-0.1249) + 0.1403 + 0.1403}{0.8597 \cdot (1 - 0.8597) + 0.1249 \cdot (1 - 0.1249) + 0.8597 \cdot (1 - 0.8597) + 0.8597 \cdot (1 - 0.8597)} \approx 0.63$

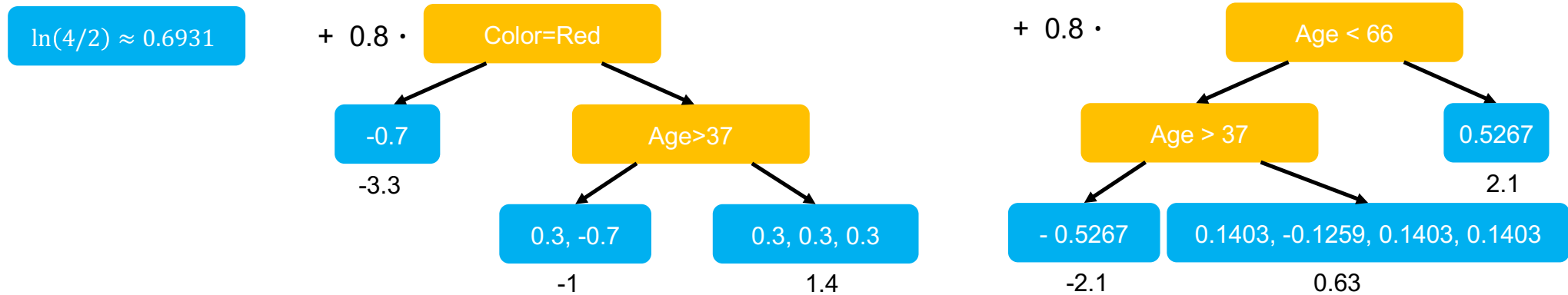


Likes Popcorn	Age	Favorite Color	Loves Aag	Pred. Prob.	Residual
Yes	12	Blue	Yes	0.8597	0.1403
Yes	87	Green	Yes	0.4733	0.5267
No	44	Blue	No	0.4733	-0.5267
Yes	19	Red	No	0.1249	-0.1249
No	32	Green	Yes	0.8597	0.1403
No	14	Blue	Yes	0.8597	0.1403



Iteration step 2: combine the models

- Combine the models



- ... and continue, until stopping criterion reached!



Exercises

Exercise 2

Gradient Boosting Classification





Multi-Class Classification

♦ Binary Classification

- Initial model: given by the log odds

♦ Multi-class Classification

- Initial model: given by the prior probability of each class
- **OneVsRest Classification**: fit one classifier per class
 - For each classifier, the class is fitted against all the other classes.
- ➔ At each iteration, one trees is constructed per class and these are combined
- ➔ Expensive for large numbers of classes



Key Takeaways

- ◆ **Boosting**
 - combine multiple low-variance/high-bias models
- ◆ **AdaBoost**
 - first boosting algorithm
 - put more “weight” on the hard-to-classify training examples
- ◆ **Gradient Boosting**
 - Compute residuals
 - Use gradient (first derivative of the residuals) for boosting
 - Trade-off between bias and variance can be tuned substantially → very good results

